

Florence: Infrastructure - Vercel

1. Overview and Purpose

Vercel serves as the primary hosting and continuous deployment platform for the Florence backend microservices and the Flutter web dashboard. It provides serverless function hosting for the Python FastAPI applications and static hosting for the compiled Flutter web output. The monorepo structure is managed via Vercel's Root Directory configuration which allows independent and automated deployments for each isolated service.

2. Configuration and Secrets Management

Team Name: Swinburne-COS40005-CTPA-FLORENCE

Team URL: `florence-dhp`

Environment variables are managed at the project level within the Vercel dashboard. They are injected into the Python serverless functions at runtime and into the Flutter web application at build time.

Data Service (ds)

Environment Variable	Description	Environments
<code>SUPABASE_URL</code>	The unique Supabase project API URL	Production, Preview
<code>SUPABASE_SERVICE_KEY</code>	Bypasses RLS for admin background tasks	Production, Preview
<code>LLM_ENGINE_SERVICE_URL</code>	Internal routing to the AI Engine	Production, Preview
<code>APP_VERSION</code>	Current application version tracker	Production, Preview

LLM Engine Service (llmes)

Environment Variable	Description	Environments
<code>LLM_API_KEY</code>	OpenRouter API key for LLM routing	Production, Preview
<code>LLM_BASE_URL</code>	OpenRouter base URL endpoint	Production, Preview
<code>LLM_MODEL</code>	The specific AI model identifier	Production, Preview
<code>DATA_SERVICE_URL</code>	Internal routing to the Data Service	Production, Preview
<code>APP_VERSION</code>	Current application version tracker	Production, Preview

LLM Chatbot Service (llmcs)

Environment Variable	Description	Environments
<code>LLM_API_KEY</code>	OpenRouter API key for LLM routing	Production, Preview
<code>LLM_BASE_URL</code>	OpenRouter base URL endpoint	Production, Preview
<code>LLM_MODEL</code>	The specific AI model identifier	Production, Preview
<code>DATA_SERVICE_URL</code>	Internal routing to the Data Service	Production, Preview
<code>APP_VERSION</code>	Current application version tracker	Production, Preview

Flutter Web Dashboard (web)

Environment Variable	Description	Environments
<code>DATA_SERVICE_URL</code>	Live backend URL for Clinician/Admin UI	Production
<code>LLM_CHATBOT_SERVICE_URL</code>	Live backend URL for Chatbot UI	Production

3. Technical Implementation Details

Project Architecture

The monorepo is split into four distinct Vercel projects to ensure that updates to one microservice do not trigger unnecessary rebuilds of the others:

- **ds (Data Service):** Handles all CRUD operations and Supabase proxying. Root directory: `florence/data_service` . Framework: FastAPI.
- **lmes (LLM Engine Service):** Handles background AI tasks including nutrition analysis and risk assessment. Root directory: `florence/llm_engine_service` . Framework: FastAPI.
- **llmcs (LLM Chatbot Service):** Powers the patient-facing conversational AI. Root directory: `florence/llm_chatbot_service` . Framework: FastAPI.
- **web (Flutter Web):** Hosts the Admin, Clinician and Patient web dashboards. Root directory: `florence/platform_service` . Framework: Custom (Build script: `bash build_web.sh` and Output directory: `build/web`).

Deployment and Branching Strategy

- **Production Environment:** Tracks the `production` branch. Merges to this branch trigger live production deployments.
- **Preview Environment:** Tracks all unassigned branches (primarily `main`). Pushes to `main` generate preview deployments (e.g. `dev-ds-florence-dhp.vercel.app`) for staging and testing.
- **Ignored Build Step:** Configured to "Automatic". Vercel will intelligently skip deployments if a commit does not contain changes to the specific project's root directory or its dependencies.

Infrastructure Settings

- **Function Region:** Asia Pacific Singapore (sin1) to ensure low latency for the target demographic.
 - **Fluid Compute:** Enabled to automatically manage concurrency and optimise performance for the Python serverless functions.
 - **Deployment Protection:** Vercel Authentication is turned off to allow the mobile application and external users to access the APIs and web dashboards without Vercel team login barriers.
 - **Routing:** Each Python microservice relies on a local `vercel.json` file to route all incoming traffic to its respective `main.py` entry point.
-

4. Billing Limits and Day 2 Operations

- **Current Tier:** Hobby (Free) Tier.
- **Limits to Watch:** The Hobby tier includes strict limits on Function Invocations (1M per month) and Fast Data Transfer (100 GB per month). Fluid Active CPU and Provisioned Memory also have monthly quotas.

- **Upgrading:** If the platform moves to a live production environment with real patients, upgrade to the Vercel Pro tier must be done to increase limits, access advanced deployment protection and enable commercial usage.
- **Adding New Microservices:** To add a new service in the future, create a new Vercel project, point it to the same Git repository and set the specific Root Directory. Ensure a `vercel.json` file is present in that directory to route traffic correctly.